

UART

The architecture diagram of the UART IP is shown below.

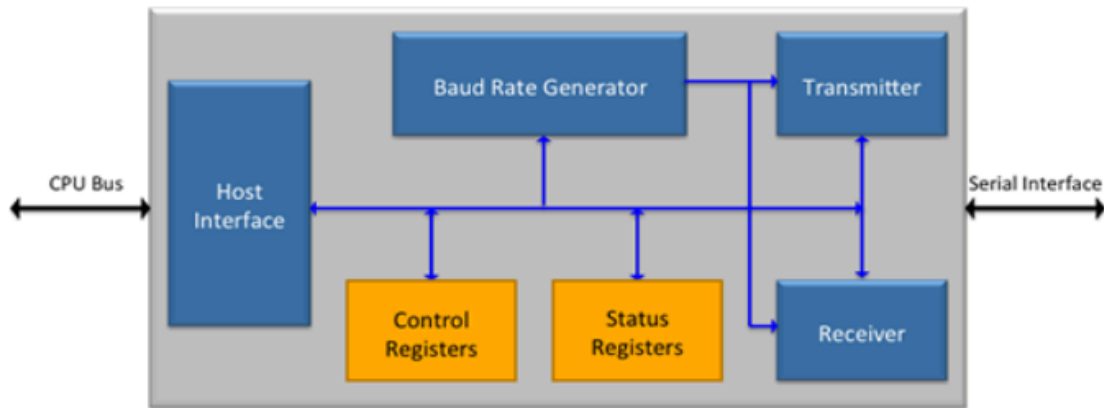
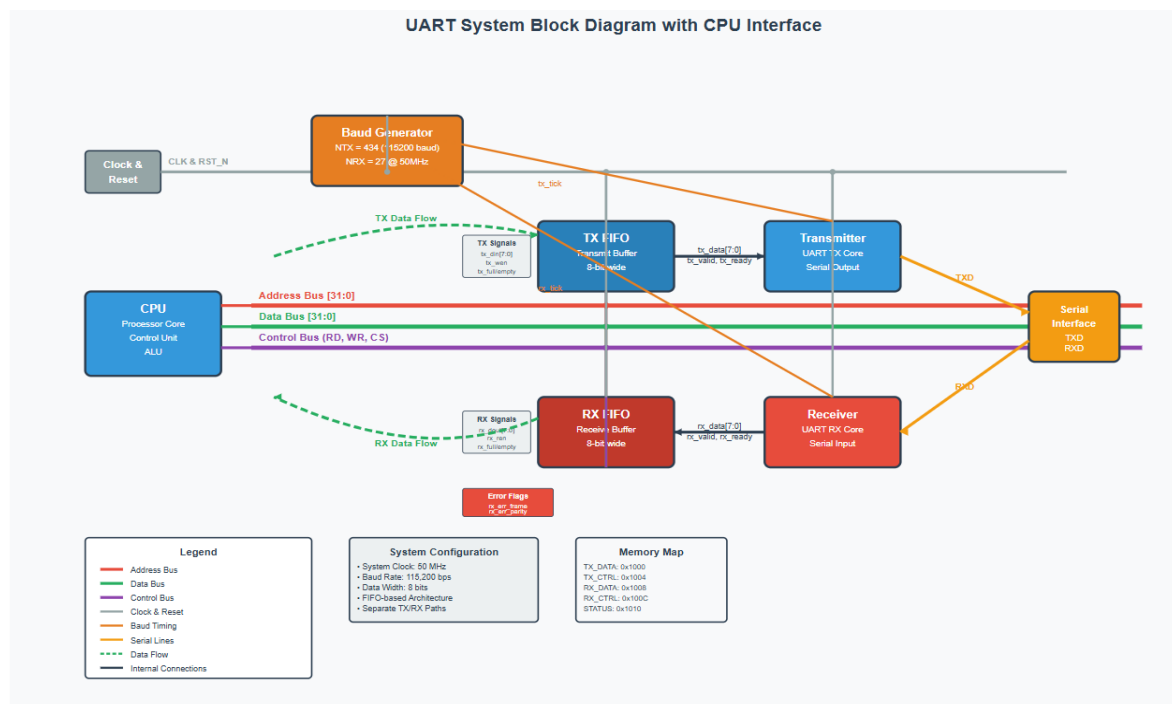
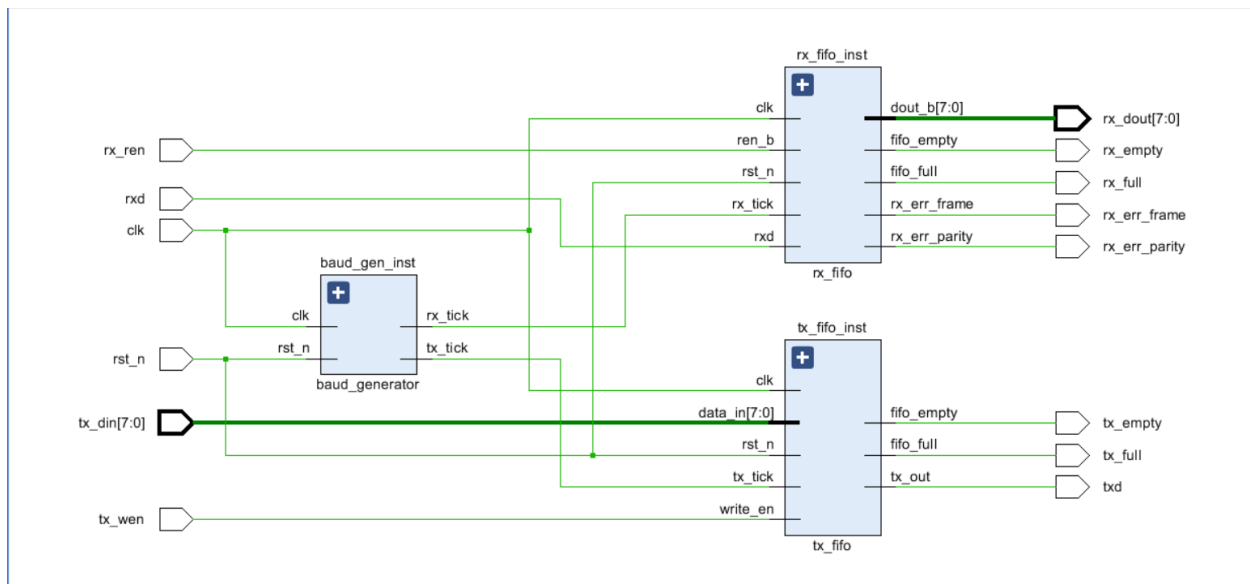


Figure 1: UART IP Block Diagram





Transmitter :

Data path

- **8-bit shift register:** holds `tx_data`, shifts right each `tx_tick`.
- **Bit counter (3 bits):** counts 0..7 data bits.
- **Stop counter (1–2):** counts stop bit ticks.

Control path (FSM)

- IDLE → START → DATA → STOP → IDLE

Outputs per state (driven on each `tx_tick`):

- **IDLE:** `txd=1`, `tx_ready=1`, `tx_busy=0`.
- **START:** drive `txd=0` for 1 tick.
- **DATA:** drive `txd = shift_reg[0]`, then shift right.
- **STOP:** drive `txd=1` for 1 (or 2) ticks, then pulse `tx_done`.

Reciver:

Control path (FSM)

- IDLE → START_check → DATA → STOP → IDLE

Control path (FSM)

• IDLE

- Watch `rx_d_sync`. If `rx_d_sync` goes 0 (edge), load `tick_cnt=0`, go to START_CHECK.

• START_CHECK

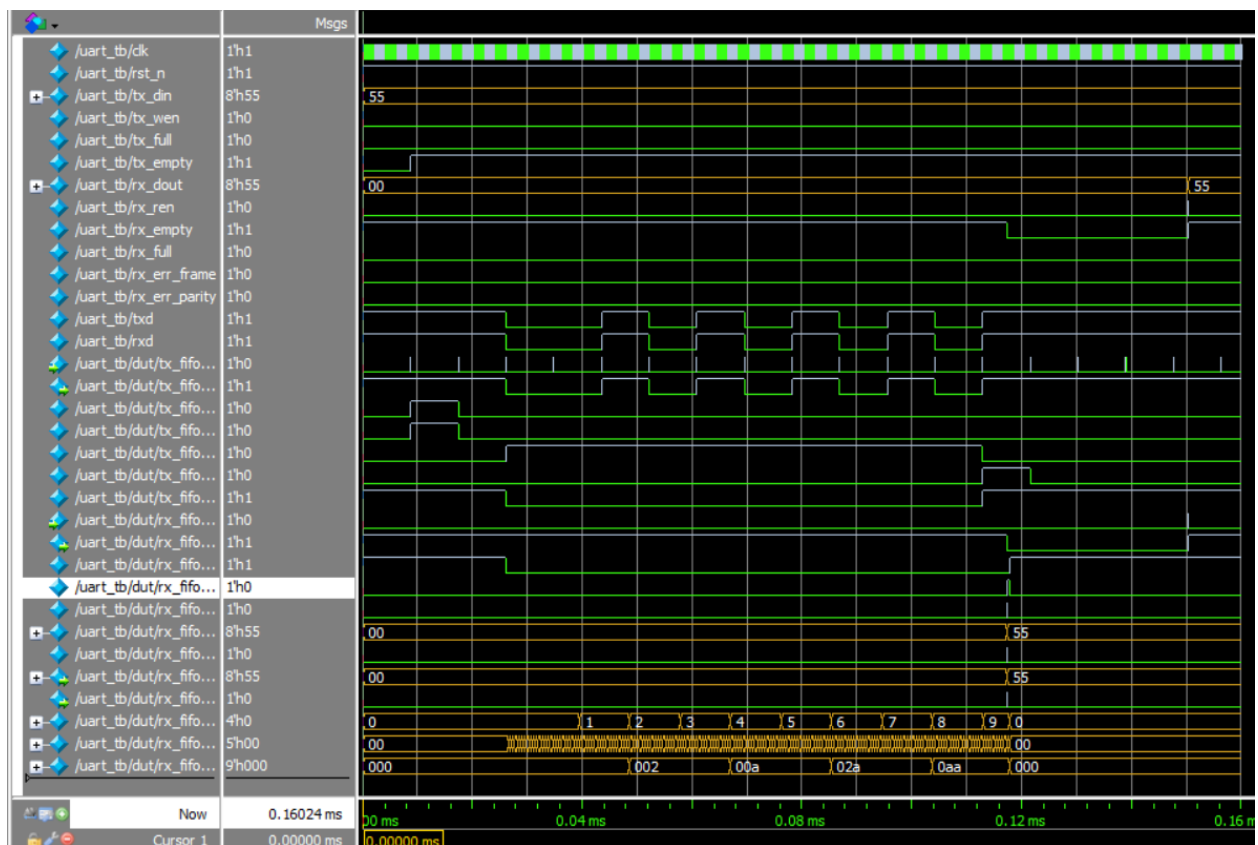
- On each `rx_tick`, increment `tick_cnt`. When `tick_cnt == OSR/2 - 1` (for OSR=16 → 7 then sample at next → we'll use 8 as center), sample `rx_d_sync`:
 - if still 0: valid start → set `tick_cnt=0`, `bit_idx=0`, go to DATA.
 - else: false start → go back to IDLE.

• DATA

- On each `rx_tick`, increment `tick_cnt`. When `tick_cnt == OSR-1` (16 ticks passed), sample center of next bit (or if you used center offset, sample when `tick_cnt == TARGET`), shift sampled bit into `rx_shift`, reset `tick_cnt=0`, increment `bit_idx`.
- When `bit_idx` reaches 8, go to STOP.

• STOP

- Wait OSR ticks (counting as before). On center sample, check `rx_d_sync`:
 - if 1 → good frame; set `rx_data = rx_shift` and `rx_valid` pulse, then go to IDLE (or DONE then IDLE).
 - if 0 → set `rx_err_frame` and (optionally) still present data or discard. Go to IDLE.



```

module uart_top (
    input wire      clk,
    input wire      rst_n,

    // TX FIFO interface
    input wire [7:0] tx_din,
    input wire      tx_wen,
    output wire     tx_full,
    output wire     tx_empty,

    // RX FIFO interface
    output wire [7:0] rx_dout,
    input wire      rx_ren,
    output wire     rx_empty,
    output wire     rx_full,
    output wire     rx_err_frame,
    output wire     rx_err_parity,

    // UART serial lines
    output wire     txd,
    input wire      rxd
);

```

```

//-----
// Baud Generator
//-----
wire tx_tick;
wire rx_tick;

baud_generator #(
    .NTX(434),      // for 115200 baud @ 50MHz
    .NRX(27)
) baud_gen_inst (
    .clk(clk),
    .rst_n(rst_n),
    .tx_tick(tx_tick),
    .rx_tick(rx_tick)
);

//-----
// TX FIFO + Transmitter
//-----
tx_fifo tx_fifo_inst (
    .clk(clk),
    .rst_n(rst_n),
    .data_in(tx_din),
    .write_en(tx_wen),
    .fifo_full(tx_full),
    .fifo_empty(tx_empty),
    .tx_tick(tx_tick),
    .tx_out(txd)
);

//-----
// RX FIFO + Receiver
//-----
rx_fifo rx_fifo_inst (
    .clk(clk),
    .rst_n(rst_n),
    .rx_d(rxd),
    .rx_tick(rx_tick),
    .dout_b(rx_dout),
    .ren_b(rx_ren),
    .fifo_empty(rx_empty),
    .fifo_full(rx_full),
    .rx_err_frame(rx_err_frame),

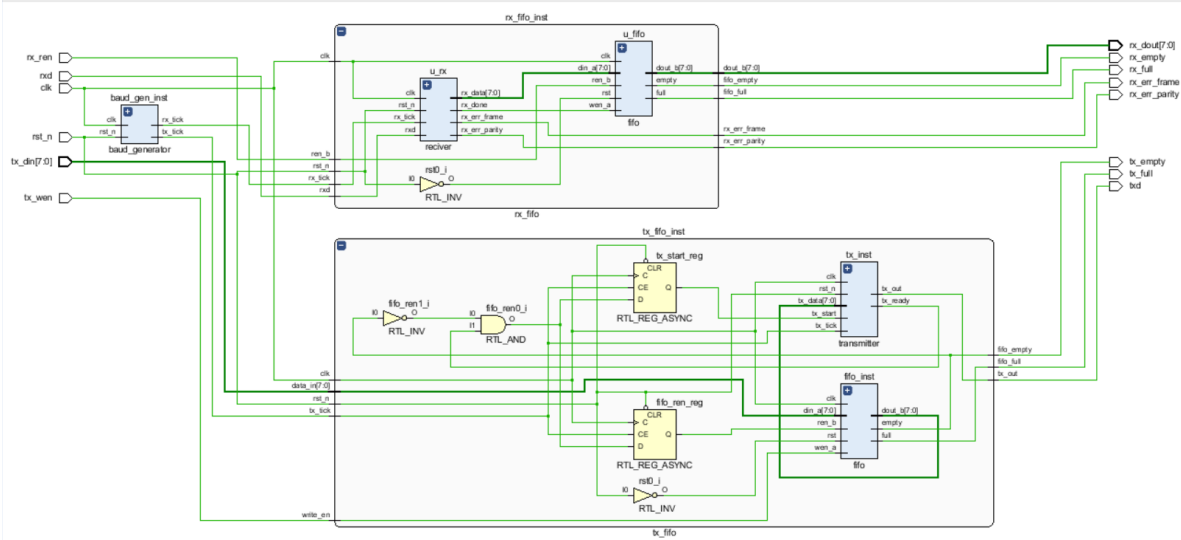
```

```

        .rx_err_parity(rx_err_parity)
    );

endmodule

```



Reciver :

```
module reciver #(
    parameter IDLE = 2'b00,
    parameter START = 2'b01,
    parameter DATA = 2'b10,
    parameter STOP = 2'b11
) (
    input clk, rx_tick, rst_n,
    input rxd,
    output reg rx_ready, // Indicates the receiver is ready to receive data
    output reg rx_valid , // Indicates that the received data is valid for
    register
    output reg [7:0] rx_data,
    output reg rx_done, // Indicates that data reception is complete and valid
    from register for fifo
    output reg rx_err_frame , // Indicates if there was a framing error
    output reg rx_err_parity // Indicates if there was a parity error
);

reg rxd_r;
reg rxd_synch; // For synchronizing the rxd signal

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        rxd_synch <= 1'b1; // Default to idle state
    end else begin
        rxd_r <= rxd;
        rxd_synch <= rxd_r; // Synchronize the rxd signal
    end
end

reg rx_valid_r1, rx_valid_r2 , rx_valid_pulse ; // Registers to create a pulse
for rx_valid
reg [1:0] cs, ns;
reg [3:0] bit_count;
reg [4:0] rx_counter;
reg [8:0] data_reg;

// State machine for receiver
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        cs <= IDLE;
        rx_ready <= 1;
```

```

    rx_valid <= 0;
    rx_done <= 0;
    bit_count <= 0;
    rx_counter <= 0;
    data_reg <= 0;
    rx_err_frame <= 0;
    rx_err_parity <= 0;
    rx_data <= 0;

end else begin
    cs <= ns;
end
end
end

// Next state logic

always @ (*)begin
    case (cs)
        IDLE : begin
            if (!rx_synch && rx_ready) begin
                ns = START ;
            end else begin
                ns = IDLE;
            end
        end
        START : begin

            if (rx_counter == 8) begin

                if (rx_synch == 1'b0) begin
                    rx_counter <= 0; // reset counter, found the middle
                    ns <= DATA; // go to data state
                end else
                    ns <= IDLE; // if not found, stay in idle
            end

        end

        DATA : begin
            if (bit_count == 9) begin
                ns = STOP;
            end else begin
                ns = DATA;
            end
        end
    end
end

```



```

    STOP : begin
    if ( rx_counter == 9 ) begin
        rx_counter = 0; // reset counter
        ns = IDLE; // After stop bit, go back to idle
    end else
        ns = STOP; // Stay in stop state until the stop bit is received

    end
    default: ns = IDLE;
endcase
end

// Output logic

always @(posedge clk ) begin
    case (cs)
        IDLE : begin
            rx_ready <= 1;
            rx_valid <= 0;
            rx_done <= 0;
            bit_count <= 0;
            rx_counter <= 0; // Reset the counter
            data_reg <= 0;
            rx_err_frame <= 0;
            rx_err_parity <= 0;
        end

        START : begin
            rx_ready <= 0; // Not ready to receive data
            if (rx_tick) rx_counter <= rx_counter + 1;
        end

        DATA : begin

            if (rx_tick) begin

                if ( rx_counter == 16 ) begin
                    data_reg[bit_count] <= rxd_synch; // Capture the data bit after
16 rx_ticks from checkin the start bit but after the 8 rx_ticks from the tx_tick
high

                    bit_count <= bit_count + 1;
                    rx_counter <= 0;
                end else begin
                    rx_counter <= rx_counter + 1; // Increment the counter

```

```

        end
    end
end
STOP : begin
    if ( rx_counter == 8 ) begin
        if (rxd_synch == 1'b1) begin // Check for stop bit
            rx_valid_pulse <= (rx_valid_r1) & ( ~ rx_valid_r2); // pulse
for one clk
            rx_valid <= 1; // Data is valid
            rx_err_frame <= 0; // No framing error
        end else begin
            rx_err_frame <= 1; // Framing error if stop bit is not high
            rx_valid <= 0;
        end
    end

    if (rx_tick) begin

        rx_counter <= rx_counter + 1; // Increment the counter

    end

    // Check for parity error
    if ( ^data_reg == 1) begin // Check parity
        rx_err_parity <= 1; // Parity error
    end else begin
        rx_err_parity <= 0; // No parity error
    end

end

default: begin
    rx_ready <= 1;
    rx_valid <= 0;
    rx_done <= 0;
    bit_count <= 0;
    data_reg <= 0;
    rx_err_frame <= 0;
    rx_err_parity <= 0;
end

endcase
end

```

```

    // store the date to register if rx_valid high

always @ (posedge clk ) begin

    if (rx_valid_pulse) begin
        rx_done <= 1; // Data reception is complete and indicate the fifo
        rx_data <= data_reg [8:1]; // Store the received data without the parity
bit
    end else begin
        rx_done <= 0;
    end
end

// register rx_valid to make pulse

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        rx_valid_r1 <= 0;
        rx_valid_r2 <= 0;
    end else begin
        rx_valid_r1 <= rx_valid;
        rx_valid_r2 <= rx_valid_r1;
    end
end

endmodule

```

Transmitter:

```
module transmitter #(
    parameter IDLE  = 2'b00,
    parameter START = 2'b01,
    parameter DATA = 2'b10,
    parameter STOP  = 2'b11
)()

    input clk, rst_n,
    input tx_tick,
    input [7:0] tx_data,
    input tx_start,
    output reg tx_busy,
    output reg tx_done,
    output reg tx_ready,
    output reg tx_out

);

reg [1:0] cs, ns;
reg [3:0] bit_count;
reg [8:0] data_reg; // extended to 9 bits for parity bit

// State machine for transmitter
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        cs <= IDLE;
        tx_busy <= 0;
        tx_done <= 0;
        tx_ready <= 1;
        tx_out <= 1; // Idle state for TX line
        bit_count <= 0;
        data_reg <= 0;
    end else if (tx_tick) begin // enable with tx_tick
        cs <= ns;
    end
end

// Next state logic
always @(*) begin
    case (cs)
        IDLE: begin
            if (tx_start && tx_ready) begin
                ns = START;
            end
        end
    endcase
end
```

```

        end else begin
            ns = IDLE;
        end
    end
START : begin
    ns = DATA;
end
DATA : begin
    if (bit_count == 8) begin
        ns = STOP;
    end else begin
        ns = DATA;
    end
end
end

STOP : begin
    if (!tx_start)
        ns = IDLE;
    else
        ns = START;
    end
default: ns = IDLE;
endcase
end

// Output logic and data handling
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        tx_busy    <= 0;
        tx_done    <= 0;
        tx_ready   <= 1;
        tx_out     <= 1;
        bit_count  <= 0;
        data_reg   <= 0;
    end else if (tx_tick) begin // enable with tx_tick
        case (cs)
            IDLE: begin
                tx_busy <= 0;
                tx_done <= 0; // pulsa done when transmission is complete
                tx_ready <= 1;
                tx_out  <= 1; // Idle state for TX line
                bit_count <= 0; // Reset bit count
            end
        end
    end
end

```

```

        START: begin
            data_reg <= {tx_data , 1'b0 }; // Load data to be transmitted
            and set parity bit to 0
            tx_busy <= 1;
            tx_ready <= 0;
            tx_out <= 0; // Start bit
            bit_count <= 0; // Reset bit count

            data_reg [0] <= ^data_reg [8:1]; // parity bit calculation
        end
        DATA: begin
            tx_out <= data_reg[bit_count]; // Transmit data bit
            bit_count <= bit_count + 1;
        end
        STOP: begin
            tx_out <= 1; // Stop bit
            tx_busy <= 0; // Transmission complete
            tx_done <= 1; // Indicate transmission done
            tx_ready <= 1; // Ready for next transmission
            bit_count <= 0; // Reset bit count for next transmission
        end
        default: begin
            tx_busy <= 0;
            tx_done <= 0;
            tx_ready <= 1;
            tx_out <= 1; // Idle state for TX line
        end
    endcase
end
end

endmodule

/*module transmitter #(
    parameter IDLE = 2'b00,
    parameter START = 2'b01,
    parameter DATA = 2'b10,
    parameter STOP = 2'b11
)(
    input tx_tick, rst_n,
    input [7:0] tx_data,
    input tx_start,
    output reg tx_busy,
    output reg tx_done,
    output reg tx_ready,

```

```

        output reg tx_out

    );

    reg [1:0] cs, ns;
    reg [3:0] bit_count;
    reg [8:0] data_reg; // extended to 9 bits for parity bit

    // State machine for transmitter
    always @(posedge tx_tick or negedge rst_n) begin
        if (!rst_n) begin
            cs <= IDLE;
            tx_busy <= 0;
            tx_done <= 0;
            tx_ready <= 1;
            tx_out <= 1; // Idle state for TX line
            bit_count <= 0;
            data_reg <= 0;
        end else begin
            cs <= ns;
        end
    end

    // Next state logic
    always @(*) begin
        case (cs)
            IDLE: begin
                if (tx_start && tx_ready) begin
                    ns = START;
                end else begin
                    ns = IDLE;
                end
            end
            START : begin
                ns = DATA;
            end
            DATA : begin
                if (bit_count == 8) begin
                    ns = STOP;
                end else begin
                    ns = DATA;
                end
            end
            STOP : begin

```

```

        if (!tx_start)
            ns = IDLE;
        else
            ns = START;
        end
        default: ns = IDLE;
    endcase
end

// Output logic and data handling

always @(posedge tx_tick) begin
    case (cs)
        IDLE: begin
            tx_busy <= 0;
            tx_done <= 0; // pulsa done when transmission is complete
            tx_ready <= 1;
            tx_out <= 1; // Idle state for TX line
            bit_count <= 0; // Reset bit count
            data_reg <= {tx_data , 1'b0 }; // Load data to be transmitted and
set parity bit to 0
        end
        START: begin
            tx_busy <= 1;
            tx_ready <= 0;
            tx_out <= 0; // Start bit
            bit_count <= 0; // Reset bit count

            data_reg [0] <= ^data_reg [8:1]; // parity bit calculation
        end
        DATA: begin
            tx_out <= data_reg[bit_count]; // Transmit data bit
            bit_count <= bit_count + 1;

        end
        STOP: begin
            tx_out <= 1; // Stop bit
            tx_busy <= 0; // Transmission complete
            tx_done <= 1; // Indicate transmission done
            tx_ready <= 1; // Ready for next transmission
            bit_count <= 0; // Reset bit count for next transmission
        end
        default: begin
            tx_busy <= 0;

```



```
        tx_done <= 0;
        tx_ready <= 1;
        tx_out <= 1; // Idle state for TX line
    end

    endcase
end

endmodule */
```